Report: Predicting Vehicle Prices Using Regression Models

This report contains the results of the analysis performed on the Beer Production dataset and the insights drawn from these analyses.

1. Data Understanding

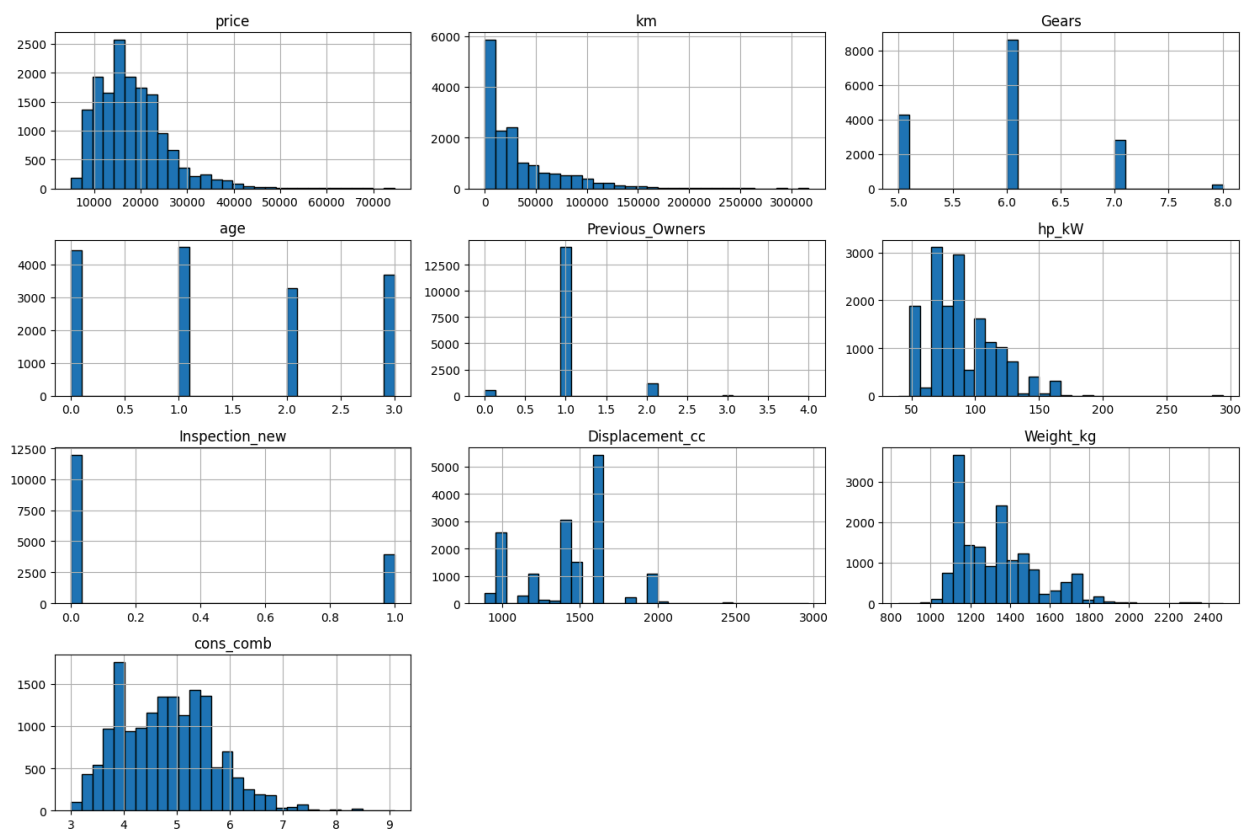
- Import the libraries and load the csv. The dataframe has `15915 rows`

2. Analysis and Feature Engineering

2.1 Preliminary Analysis and Frequency Distributions

2.1.1 Missing Values: There are no missing values

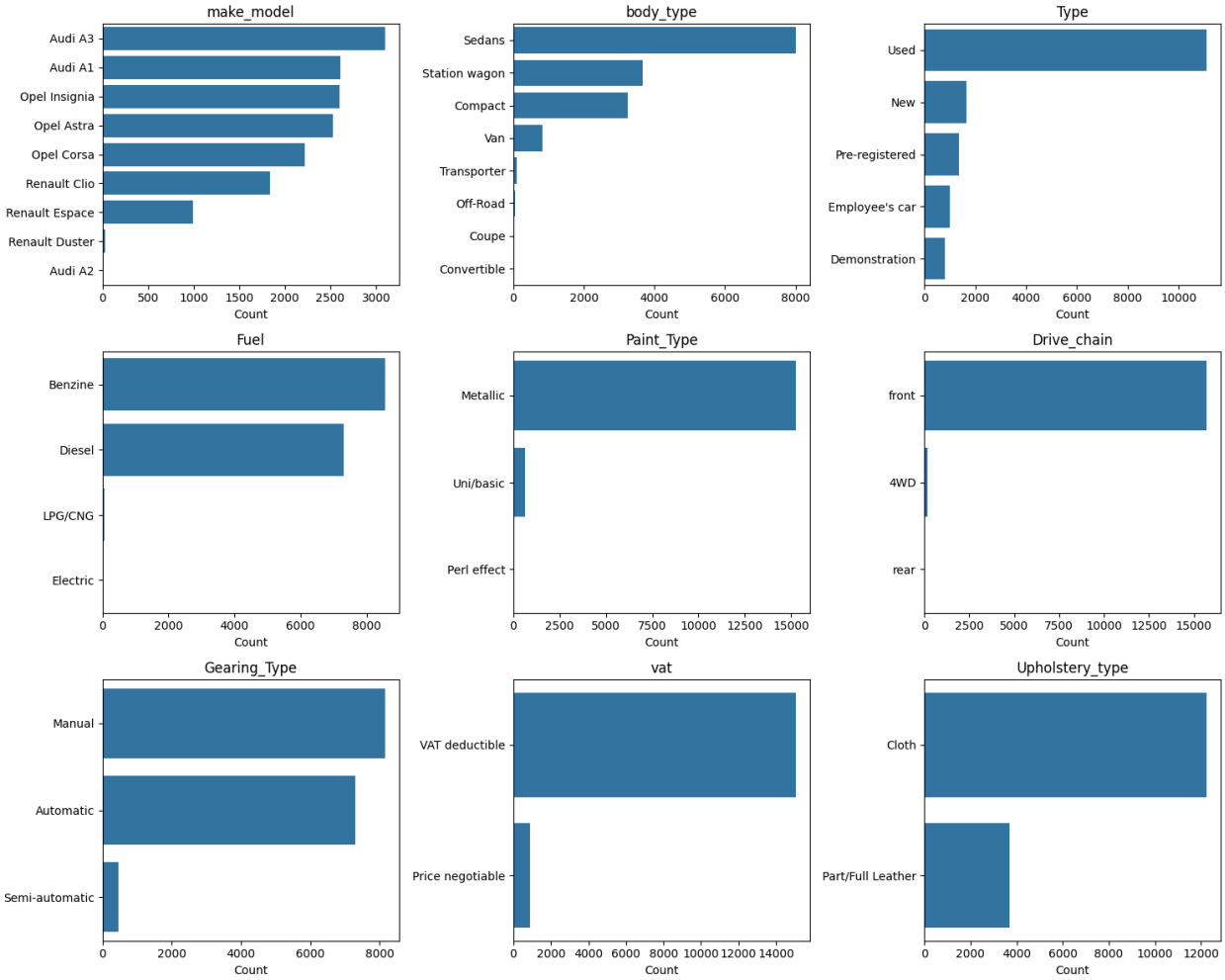
2.1.2 Numerical predictors and plot their frequency distributions.



- Many numerical features like price, km, hp_kW, Displacement_cc, Weight_kg, cons_comb exhibit right-skewed distributions.
- Gears have low cardinality, predominantly showing values of 5, 6, or 7.
- Inspection_new is a binary feature (0 or 1).

- age and Previous_Owners are discrete numerical features with limited ranges, also showing skewed distributions.

2.1.3. Identify categorical predictors and plot their frequency distributions.



- High cardinality columns (nunique > 100): ['Comfort_Convenience', 'Safety_Security', 'Extras', 'Entertainment_Media']
- These cannot be treated as NORMAL categorical columns directly.
- Consider exploding (if comma-separated), label encoding, or dropping them before modelling.
- Columns suitable for categorical analysis: ['make_model', 'body_type', 'Type', 'Fuel', 'Paint_Type', 'Drive_chain', 'Gearing_Type', 'vat', 'Upholstery_type']

Look carefully at the values stored in columns ["Comfort_Convenience", "Entertainment_Media", "Extras", "Safety_Security"].

Should they be considered categorical? Should they be dropped or handled any other way?

- As we have seen in above cells, these columns have high cardinality

    - Comfort_Convenience -> 6196
    - Safety_Security -> 4442
    - Extras -> 659
    - Entertainment_Media -> 346
- And Each value is a comma-separated list of features, e.g. "Air conditioning, Cruise control, Navigation system". So each row represent a set of features (multi-label), not a single category.
- One-hot encoding them directly would create thousands of meaningless dummy columns (one per unique combination)
- A categorical column assumes a fixed set of mutually exclusive classes — these columns violate that assumption entirely
- **So, they should NOT be treated as standard (single-label) categorical variables, but rather as multi-label categorical features requiring special encoding.**

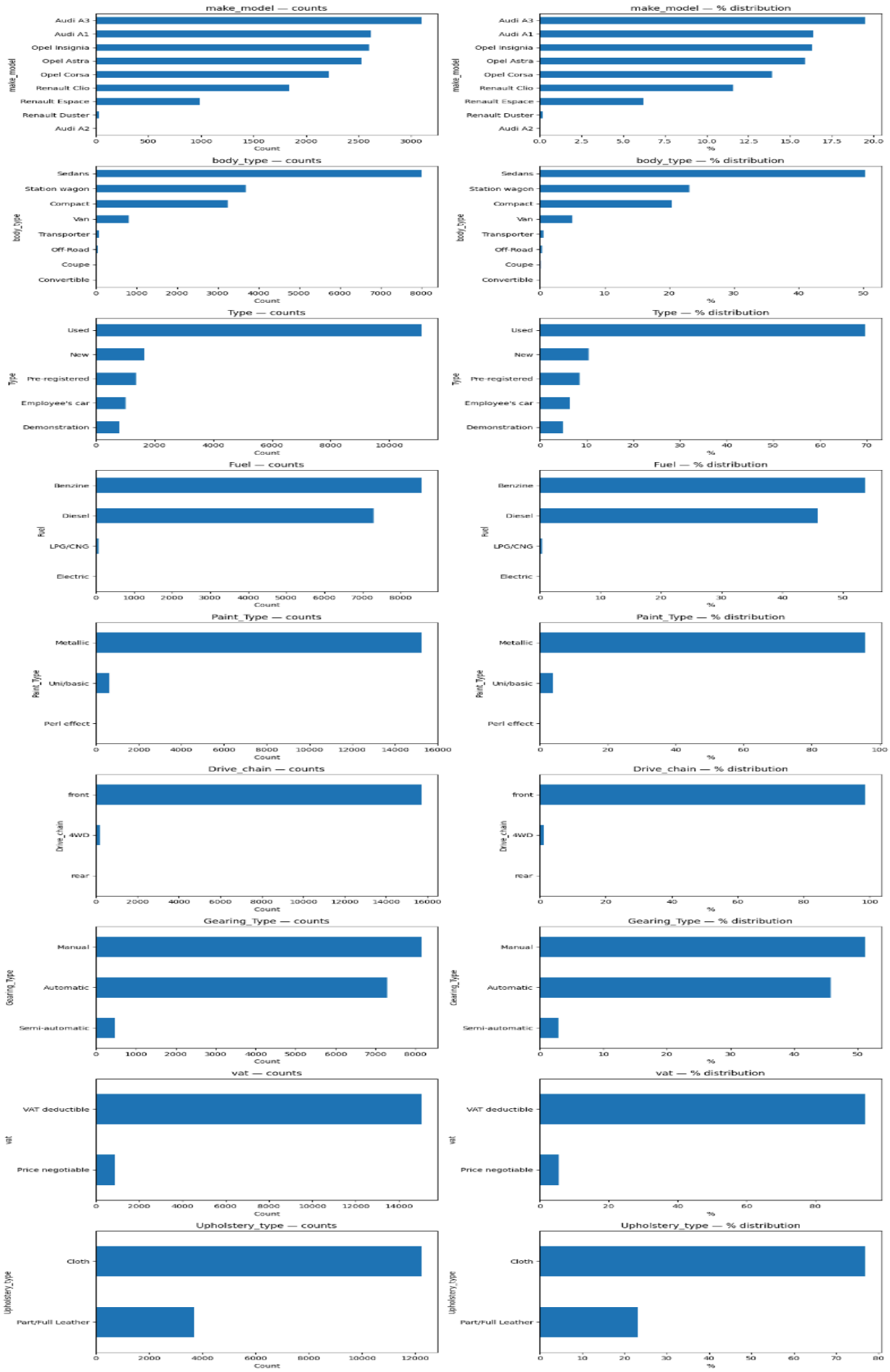
Should they be dropped or handled any other way?

- Dropping will lead to loss of information
- Another option could be to count the features: Instead of what features a car has, just **count how many it has**. The intuition is simple that a car with 10 comfort features is probably more expensive than one with 2, regardless of exactly which features they are.** We turn an unusable text column into a useful numeric one
- **In this assignment I will Binarize (one column per feature)**

2.1.4 Fix columns with low frequency values and class imbalances.

- Columns where one value has more than 80% values are flagged as imbalance
- Rare values are merged into Other

```
STEP 1: Grouping rare categories
==================================================

make_model: 9 -> 8 categories
  Merged into 'Other': ['Renault Duster', 'Audi A2']

body_type: 8 -> 5 categories
  Merged into 'Other': ['Transporter', 'Off-Road', 'Coupe', 'Convertible']

Fuel: 4 -> 3 categories
  Merged into 'Other': ['LPG/CNG', 'Electric']

Paint_Type: 3 -> 3 categories
  Merged into 'Other': ['Perl effect']


STEP 2: Merging similar Type categories
==================================================
Type
Used             11095
New               3013
Demonstration     1807
Name: count, dtype: int64

Merged 'Pre-registered' -> 'New'   (both new-like cars)
Merged 'Employee's car' -> 'Demonstration'   (both short use, short distance)


STEP 3: Evaluating highly imbalanced columns
==================================================

Drive_chain distribution:
Drive_chain
front    0.986931
4WD      0.012818
rear     0.000251
Name: proportion, dtype: float64
```


make_model — counts
Audi A3
Audi A1
Opel Insignia
Opel Astra
Opel Corsa
Renault Clio
Renault Espace
Renault Duster
Audi A2
make_model
Count
make_model — % distribution
%
body_type — counts
Sedans
Station wagon
Compact
Van
Transporter
Off-Road
Coupe
Convertible
body_type
Count
body_type — % distribution
%
Type — counts
Used
New
Pre-registered
Employee's car
Demonstration
Type
Count
Type — % distribution
%
Fuel — counts
Benzine
Diesel
LPG/CNG
Electric
Fuel
Count
Fuel — % distribution
%
Paint_Type — counts
Metallic
Uni/basic
Perl effect
Paint_Type
Count
Paint_Type — % distribution
%
Drive_chain — counts
front
4WD
rear
Drive_chain
Count
Drive_chain — % distribution
%
Gearing_Type — counts
Manual
Automatic
Semi-automatic
Gearing_Type
Count
Gearing_Type — % distribution
%
vat — counts
VAT deductible
Price negotiable
vat
Count
vat — % distribution
%
Upholstery_type — counts
Cloth
Part/Full Leather
Upholstery_type
Count
Upholstery_type — % distribution
%

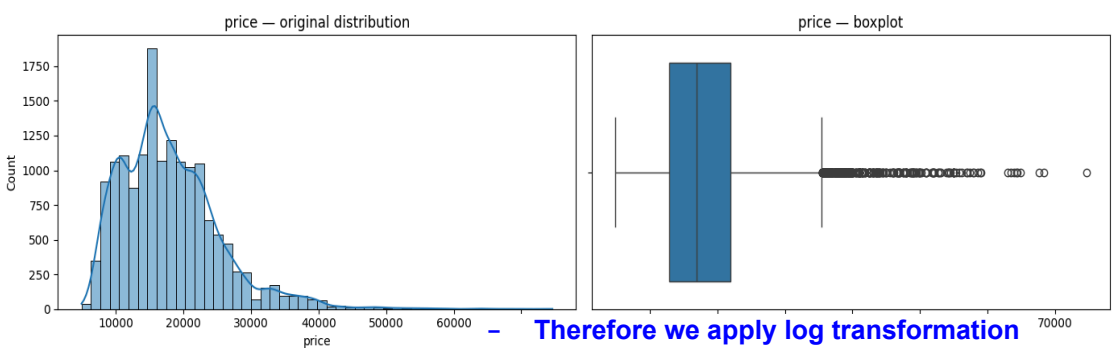
Summary of issues found:

Column	Issue	Fix
make_model	Renault Duster 0.2%, Audi A2 0.0%	Group rare into Other
body_type	Transporter 0.6%, Off-Road 0.4%, Coupe 0.2%, Convertible 0.1%	Group rare into Other
Type	Pre-registered ≈ New, Employee's car ≈ Demonstration	Merge similar categories
Fuel	LPG/CNG 0.4%, Electric 0.0%	Group rare into Other
Paint_Type	Perl effect 0.2%, dominated by Metallic 95.8%	Group rare + flag imbalance
Drive_chain	rear 0.0%, dominated by front 98.7%	Review
Gearing_Type	Reasonable distribution	No fix needed
vat	Dominated by VAT deductible 94.5%	Review
Upholstery_type	Reasonable split 77%/23%	No fix needed

2.1.5 Identify target variable and plot the frequency distributions. Apply necessary transformations.

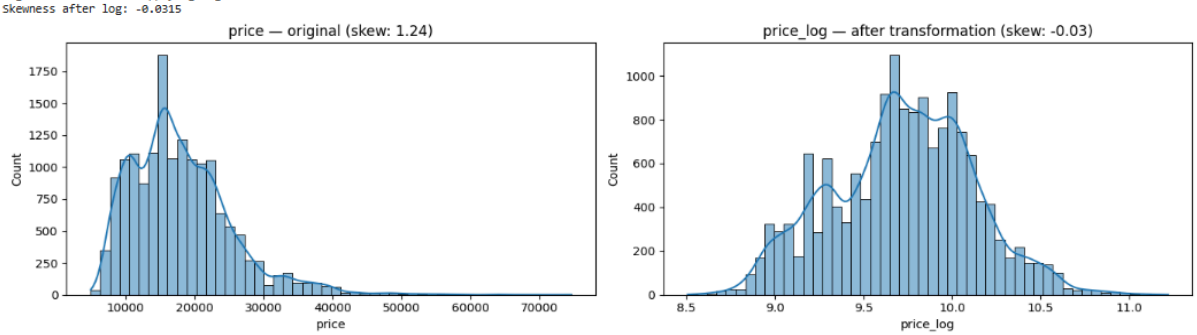
- Price is the target variable.
- It is skewed towards right

-



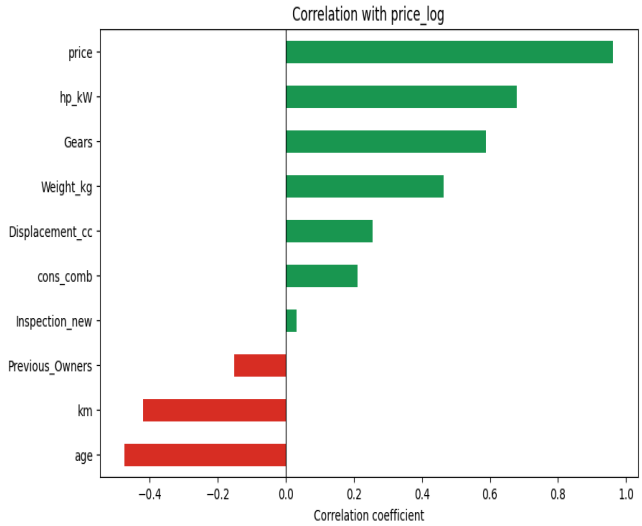
– **Therefore we apply log transformation**

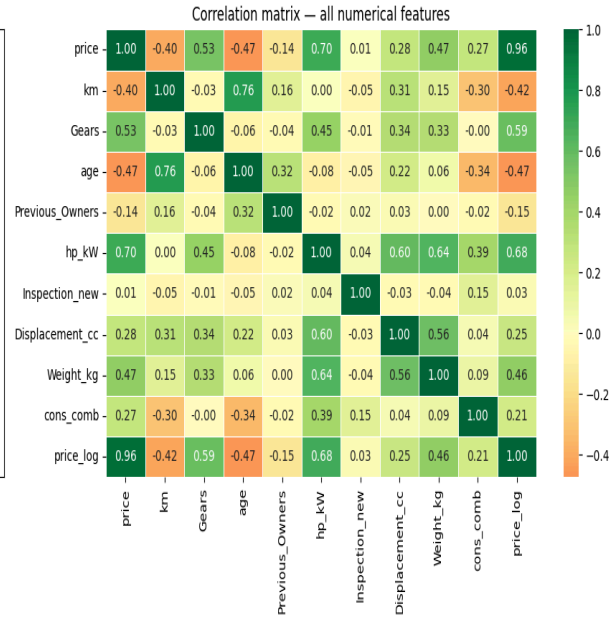
```
Skewness before transformation: 1.2362
High skew detected — applying log transformation
Skewness after log: -0.0315
```



2.2 Correlation analysis

2.2.1 Plot the correlation map between features and target variable.





Primary Price Drivers: The map reveals that **`hp_kW` (Horsepower)** and **`Weight_kg`** have the strongest positive correlations with price. This indicates that performance-oriented and larger vehicles consistently command higher market premiums.

The Depreciation Effect: **`age`** and **`km` (Kilometers)** show the strongest negative correlations. This confirms that vehicle age is the most aggressive "price killer," with mileage acting as a secondary but significant factor in value loss.

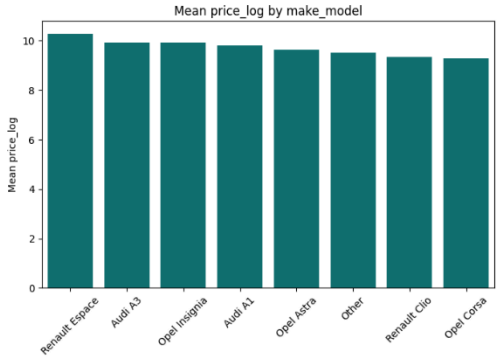
High Multicollinearity: Several "hot spots" between independent variables—such as **`Displacement_cc`** and **`hp_kW`**—highlight redundant information. This explains why a simple Linear Regression might struggle with coefficient stability compared to Lasso.

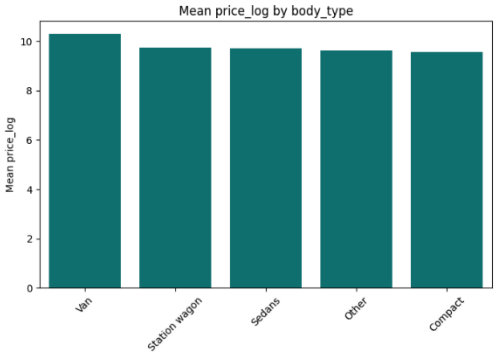
Lower Impact Features: Physical attributes like **`Nr_of_Doors`** and **`Nr_of_Seats`** show correlations near zero. This suggests that while these features are necessary for listing, they do not fundamentally drive the car's valuation in this dataset.

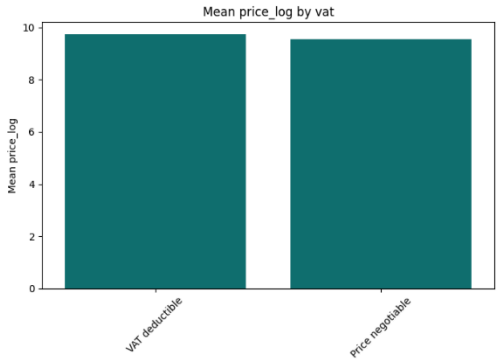
Validation of Log-Transformation: The clear linear gradients between the features and the **`log_price`** target validate the decision to use a log-transform, as it linearized the relationships and made the correlations more distinct and reliable for modeling.

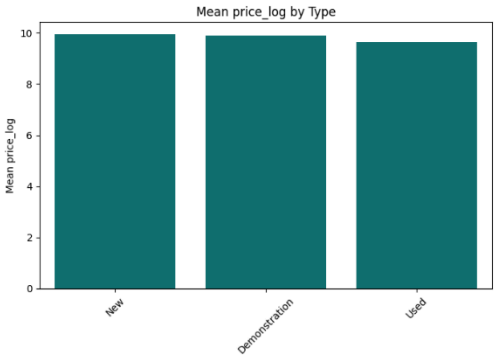
2.2.2 Analyse correlation between categorical features and target variable.

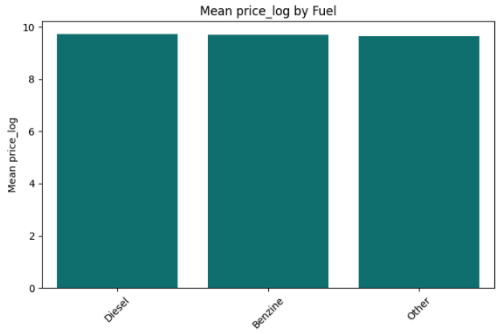
- We use price_log as target to analyse the correlation

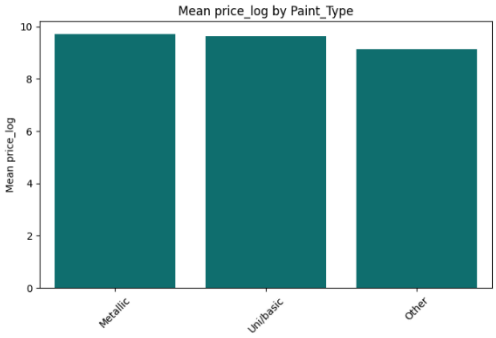

Mean price_log by make_model
Mean price_log
Renault Espace
Audi A3
Opel Insignia
Audi A1
Opel Astra
Other
Renault Clio
Opel Corsa

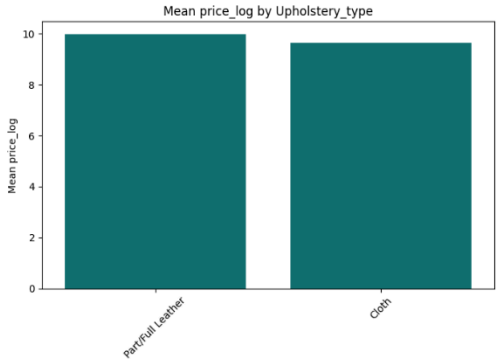

Mean price_log by body_type
Mean price_log
Van
Station wagon
Sedans
Other
Compact

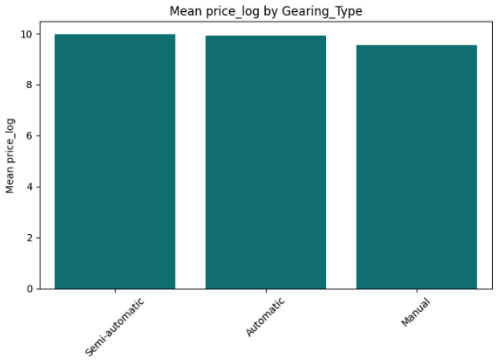

Mean price_log by vat
Mean price_log
VAT deductible
Price negotiable

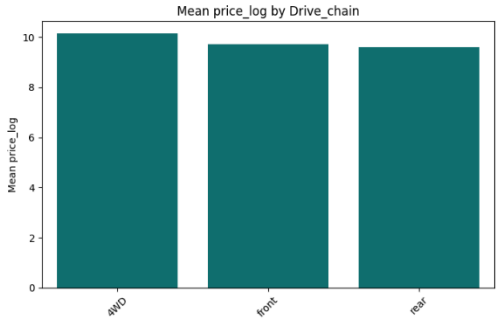

Mean price_log by Type
Mean price_log
New
Demonstration
Used


Mean price_log by Fuel
Mean price_log
Diesel
Benzine
Other


Mean price_log by Paint_Type
Mean price_log
Metallic
Uni/basic
Other


Mean price_log by Upholstery_type
Mean price_log
Part/full Leather
Cloth


Mean price_log by Gearing_Type
Mean price_log
Semi-automatic
Automatic
Manual

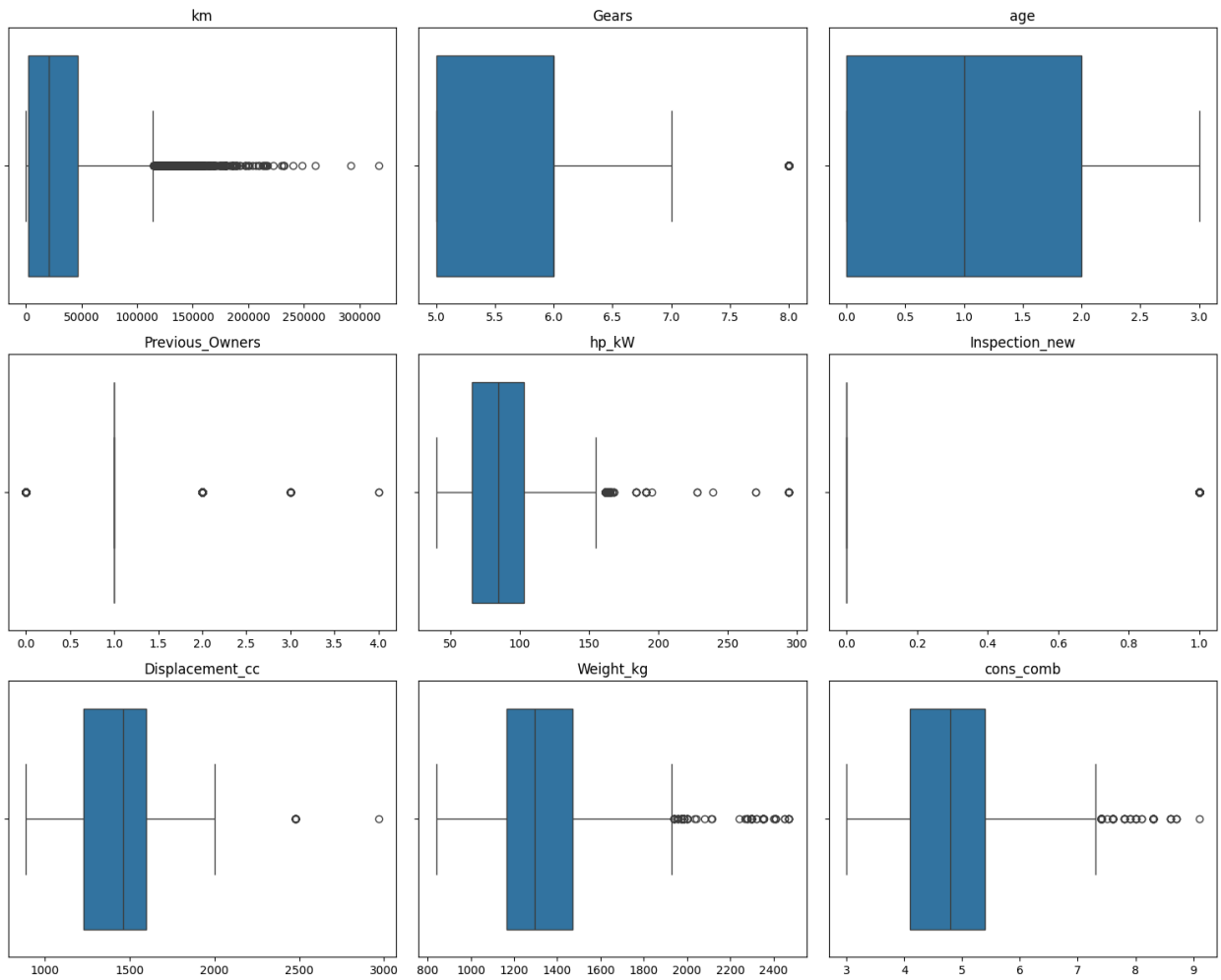

Mean price_log by Drive_chain
Mean price_log
4WD
front
rear

2.3 Outlier analysis

2.3.1 Identify potential outliers in the data.

```
OUTLIER SUMMARY (IQR method)
=====================================================
```

column	Q1	Q3	IQR	lower_bound	upper_bound	n_outliers	pct
Inspection_new	0.0	0.0	0.0	0.00	0.00	3932	24.71
Previous_Owners	1.0	1.0	0.0	1.00	1.00	1757	11.04
km	1920.5	46900.0	44979.5	-65548.75	114369.25	689	4.33
hp_kW	66.0	103.0	37.0	10.50	158.50	361	2.27
Gears	5.0	6.0	1.0	3.50	7.50	225	1.41
cons_comb	4.1	5.4	1.3	2.15	7.35	125	0.79
Weight_kg	1165.0	1472.0	307.0	704.50	1932.50	87	0.55
Displacement_cc	1229.0	1598.0	369.0	675.50	2151.50	21	0.13
age	0.0	2.0	2.0	-3.00	5.00	0	0.00



Outlier Analysis Summary

The IQR method was applied to all numerical columns to identify outliers. The results are interpreted as follows:

Columns where IQR method is not applicable

- **`Inspection_new`**: Binary column (0/1). IQR = 0, so any value of 1 is flagged as outlier (24.71%). This is a false alarm — value counts confirm a valid binary distribution (75.3% vs 24.7%). No action needed.
- **`Previous_Owners`**: Low cardinality column (values 0-4). IQR = 0, causing the method to break down (11.04% flagged). Value counts confirm a sensible distribution — majority have 1 previous owner, with a small number having 2, 3, or 4. No action needed.

Columns with real outliers requiring treatment

- **`km`**: 4.33% of values exceed the upper bound of 114,369 km. These are genuinely high mileage cars that could distort the model. Will be capped using IQR bounds (Winsorization).
- **`hp_kW`**: 2.27% of values exceed the upper bound of 158.5 kW. These are high performance cars. Will be capped using IQR bounds.

Columns with negligible outliers — no action needed

- **`Gears`** (1.41%), **`cons_comb`** (0.79%), **`Weight_kg`** (0.55%), **`Displacement_cc`** (0.13%) all have under 1.5% outliers. These likely represent legitimate high-end cars carrying real price signal. Capping these would remove useful information from the model.
- **`age`**: No outliers detected.

2.3.2 Handle the outliers suitably.

We use Winsorization to handle outliers by clipping between lower and outer range

```
km — clipped to [-65548.8, 114369.2]
  max before: 317000.0  →  max after: 114369.2
  min before: 0.0  →  min after: 0.0

hp_kW — clipped to [10.5, 158.5]
  max before: 294.0  →  max after: 158.5
  min before: 40.0  →  min after: 40.0
```

2.4 Feature Engineering

2.4.1 Fix any redundant columns and create new ones if needed.

Created new columns:

1. power to weight ratio — performance indicator
    - df['power_to_weight'] = df['hp_kW'] / df['Weight_kg']
2. km per year — usage intensity, more informative than km alone
    - df['km_per_year'] = df['km'] / (df['age'] + 1)

2.4.2 Analysis and feature engineering on `['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security']`.

Total unique features across all columns:

- Comfort_Convenience — 38
- Safety_Security — 29
- Extras — 17
- Entertainment_Media — 10

Total = 94 binary columns if we binarize everything. That's manageable but we should remove low variance ones first.

Features to drop:

1. Comfort_Convenience — 12 features to drop (Parking assist self-steering, Panorama roof, Heads-up display, Sunroof, Massage seats, Auxiliary heating, 3. Air suspension, Leather seats, Wind deflector, Windshield, Electric Starter + Air conditioning is 94.8% — borderline keep)
2. Entertainment_Media — 1 feature to drop (Television)
3. Extras — 9 features to drop
4. Safety_Security — 2 features to drop

No features above 95% — so nothing to drop from the top end except ABS at 94.4% and Air conditioning at 94.8% — borderline, keep them as they still have ~5% variance.

==

```
Comfort_Convenience: 27 features binarized

Entertainment_Media: 9 features binarized

Extras: 8 features binarized

Safety_Security: 27 features binarized
```

```
Total new binary columns: 71
Shape after binarization: (15915, 93)
==
```

Top Features

```
Safety_Security__LED Headlights                              0.436199
Comfort_Convenience__Automatic climate control               0.414838
Safety_Security__Lane departure warning system               0.378763
Safety_Security__Blind spot monitor                          0.377029
Comfort_Convenience__Park Distance Control                   0.371756
Safety_Security__Traffic sign recognition                    0.354418
Comfort_Convenience__Rain sensor                             0.351564
Safety_Security__Emergency brake assistant                   0.344904
Comfort_Convenience__Parking assist system sensors front     0.332329
Comfort_Convenience__Navigation system                       0.329263
```

2.4.3 Perform feature encoding.

1. Type encoded: {0: 11095, 2: 3013, 1: 1807} where {'New': 2, 'Demonstration': 1, 'Used': 0}
2. Gearing_Type encoded: {0: 8149, 1: 7297, 2: 469} where {'Semi-automatic': 2, 'Automatic': 1, 'Manual': 0}
3. Upholstery_type encoded: {0: 12234, 1: 3681} where {'Part/Full Leather': 1, 'Cloth': 0}
4. One hot encoding done on ['Fuel', 'body_type', 'make_model', 'vat', 'Paint_Type', 'Drive_chain']

2.4.4 Split the data into training and testing sets.

- Features are obtained by dropping Price and Price_log
- Target is Price log.
- Split Train: Test = 80:20, random_state =42

```
X shape: (15915, 103)
y shape: (15915,)
X_train: (12732, 103)
X_test:  (3183, 103)
y_train: (12732,)
y_test:  (3183,)
```

2.4.5. Scale the features

- We scale for equalizing feature influence (bring all features to common “playing field”)
- Also because Regularization works by adding a penalty based on the magnitude of the coefficients. If features aren't scaled, a feature with a small range will naturally have a larger coefficient to compensate. The penalty would then unfairly target that specific feature. Scaling ensures the penalty is applied uniformly across all features.
- And for optimization algorithms like Gradient Descent, having features on the same scale creates a more "circular" and symmetric error landscape. This allows the algorithm to reach the optimal solution (the bottom of the hill) much faster and more directly.

I use **StandardScaler** instead of **MinMaxScaler** because unlike MinMaxScaler (which squishes all data into a tiny 0-1 range), StandardScaler does not have a fixed bounding box. If you have an outlier, StandardScaler will place it many standard deviations away from the mean, but it won't "collapse" the rest of your data into an unreadable cluster.

Just for demo, showing

```
Effect of outliers on scaling methods:
=======================================================

km:
  MinMaxScaler   — 75% of values compressed into 0.00 to 0.41
  StandardScaler — 75% of values spread  across -2.00 to 0.48

hp_kW:
  MinMaxScaler   — 75% of values compressed into 0.00 to 0.53
  StandardScaler — 75% of values spread  across -2.00 to 0.58
```

---

3. Linear Regression Models

3.1 Baseline Linear Regression Model

3.1.1. Build and fit a basic linear regression model. Perform evaluation using suitable metrics.

Sample predictions vs actual (train):

	Actual Train	Predicted Train	Difference
7704	10.19	10.13	0.06
1605	10.04	10.05	-0.01
14322	9.21	9.22	-0.01
13630	8.97	8.97	0.01
13922	9.14	9.00	0.14

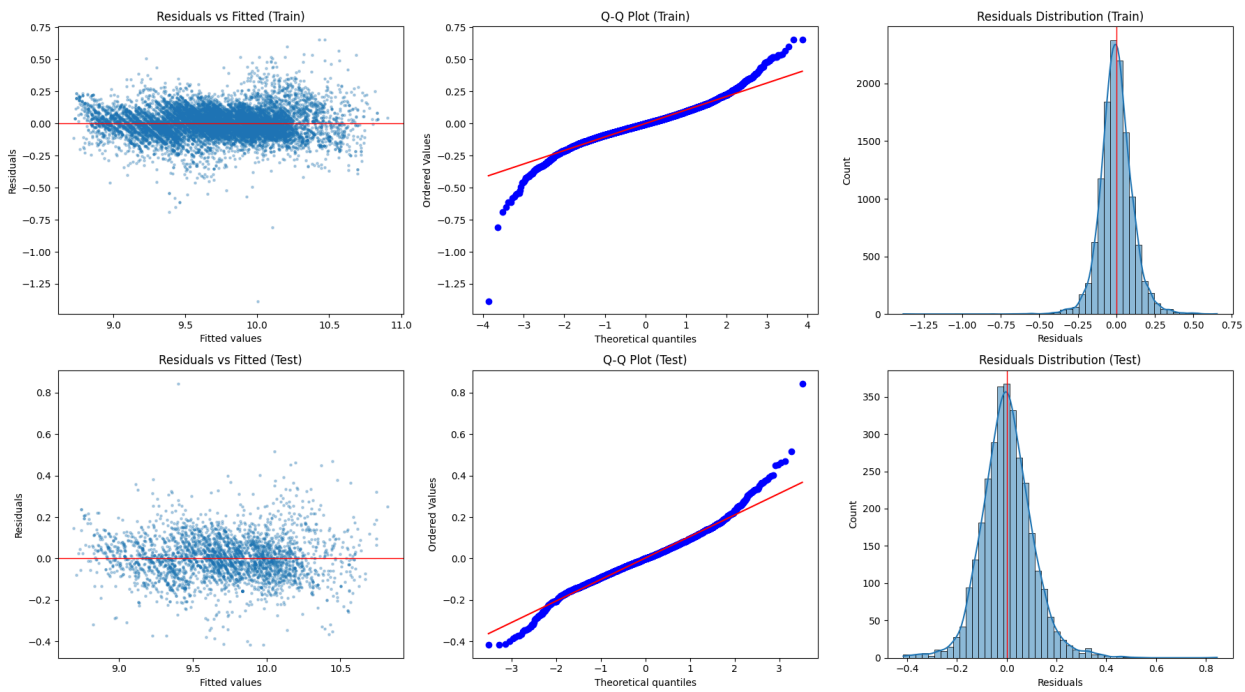
Sample predictions vs actual (test):

	Actual Test	Predicted Test	Difference
13868	9.83	9.96	-0.14
8887	9.62	9.36	0.26
10255	9.28	9.34	-0.06
11766	9.72	9.82	-0.10
5489	10.08	10.05	0.03

```
LINEAR REGRESSION — Evaluation
========================================

Train
  R²   : 0.9275
  RMSE : 0.1071
  MAE  : 0.0785

Test
  R²   : 0.9310
  RMSE : 0.1050
  MAE  : 0.0782

Overfitting check:
  R² Train : 0.9275
  R² Test  : 0.9310
  Gap      : -0.0035
  ✓ Small gap — model generalises well
```

3.1.2 Analyse residuals and check other assumptions of linear regression.

```
Residuals Summary
========================================

Train:
  Mean    : -0.0000   (should be ~0)
  Std     : 0.1071
  Skew    : -0.0672   (should be ~0)
  Kurt    : 5.0663   (should be ~0)

Test:
  Mean    : 0.0024   (should be ~0)
  Std     : 0.1050
  Skew    : 0.3792   (should be ~0)
  Kurt    : 2.8924   (should be ~0)
```



1. Linearity & Homoscedasticity — Residuals vs Fitted

The residuals vs fitted plot actually shows a reasonably even horizontal band across most of the range. Overall the homoscedasticity assumption is reasonably well satisfied — no strong funnel shape, no systematic pattern.

2. Normality — Q-Q Plot

Both plots follow the diagonal line well in the middle range but deviate at the tails — points curve away at both ends. This is classic heavy-tailed behaviour meaning extreme residuals are larger than a perfect normal distribution would predict. Confirmed by kurtosis:

Train Kurt = 5.13 ← heavy tails (normal = 0)

Test Kurt = 2.49 ← moderate heavy tails

3.Normality — Residuals Distribution

Both distributions are centred at 0 — confirmed by:

Train Mean = -0.0000

Test Mean = 0.0025

VIF:

```
Iterative VIF removal (threshold = 10.0)
========================================================

Step 1: Dropping 'power_to_weight' — VIF = 91.12

Step 2: Dropping 'km' — VIF = 20.73

No removable features remain above threshold.
Stopping because remaining high-VIF features are protected.

Features removed:
['power_to_weight', 'km']

Shape before VIF: (12732, 103)
Shape after VIF : (12732, 101)

Final VIF Scores
========================================================
                                                  feature       VIF
                              make_model_Renault Espace 10.198923
                                                   hp_kW  8.536177
                                           body_type_Van  6.858337
                               make_model_Opel Insignia  6.833274
                                        Displacement_cc  6.817262
                                           Fuel_Diesel  6.811694
                                   make_model_Opel Corsa  5.220948
 Comfort_Convenience__Parking assist system sensors rear  4.970139
                                             Weight_kg  4.733210
```

body_type_Station wagon	4.563141
cons_comb	4.397762
make_model_Opel Astra	4.250990
Comfort_Convenience__Park Distance Control	4.174811
make_model_Audi A3	3.565287
make_model_Renault Clio	3.549493
age	3.014425
Comfort_Convenience__Heated steering wheel	2.835946
Comfort_Convenience__Rain sensor	2.788598
Gears	2.782253
Comfort_Convenience__Light sensor	2.730432
Extras__Roof rack	2.670238
Safety_Security__Head airbag	2.613673
Comfort_Convenience__Seat heating	2.612876
body_type_Sedans	2.586410
Safety_Security__Lane departure warning system	2.560500
Safety_Security__Central door lock with remote control	2.524742
Comfort_Convenience__Parking assist system camera	2.385837
Safety_Security__Traffic sign recognition	2.381097
km_per_year	2.363664
Comfort_Convenience__Armrest	2.334551
Safety_Security__Passenger-side airbag	2.329381
Gearing_Type	2.310563
Comfort_Convenience__Split rear seats	2.259724
Comfort_Convenience__Parking assist system sensors front	2.207280
Comfort_Convenience__Automatic climate control	2.145325
Safety_Security__Blind spot monitor	2.103665
Safety_Security__Driver-side airbag	2.090631
Comfort_Convenience__Leather steering wheel	2.055370
Type	2.042740
Comfort_Convenience__Electrical side mirrors	2.008016
Safety_Security__Emergency brake assistant	1.989938
Entertainment_Media__Hands-free equipment	1.975926
Safety_Security__Tire pressure monitoring system	1.958605
Safety_Security__LED Headlights	1.957409
Safety_Security__Daytime running lights	1.925691
Extras__Touch screen	1.915600
Safety_Security__Xenon headlights	1.860302
Comfort_Convenience__Lumbar support	1.857236
Comfort_Convenience__Keyless central door lock	1.830821
Comfort_Convenience__Navigation system	1.829924
Safety_Security__LED Daytime Running Lights	1.810200
make_model_Other	1.808539
Safety_Security__Central door lock	1.791719
Comfort_Convenience__Multi-function steering wheel	1.782599
Comfort_Convenience__Start-stop system	1.777125
Entertainment_Media__USB	1.768380

```
                                Safety_Security__Side airbag  1.753068
                              Entertainment_Media__Bluetooth  1.733191
                     Comfort_Convenience__Electric tailgate  1.721324
                       Safety_Security__Adaptive headlights  1.715795
         Comfort_Convenience__Electrically adjustable seats  1.674400
                        Comfort_Convenience__Cruise control  1.670810
                                    Safety_Security__Isofix  1.666443
                              Entertainment_Media__CD player  1.665620
                            Comfort_Convenience__Hill Holder  1.662010
                          Comfort_Convenience__Power windows  1.656951
              Safety_Security__Electronic stability control  1.647366
                                      Extras__Sport seats  1.626146
                                Safety_Security__Immobilizer  1.625659
                                     Extras__Voice Control  1.625558
                      Comfort_Convenience__Seat ventilation  1.625123
                        Comfort_Convenience__Tinted windows  1.621171
                                           body_type_Other  1.604914
                                           Upholstery_type  1.580453
                                Safety_Security__Rear airbag  1.554604
                    Safety_Security__Adaptive Cruise Control  1.550315
                                     Safety_Security__ABS  1.542738
                              Safety_Security__Alarm system  1.531076
                                  Extras__Sport suspension  1.512542
                           Safety_Security__Traction control  1.497696
                             Safety_Security__Power steering  1.451990
                                 Safety_Security__Fog lights  1.446728
                      Entertainment_Media__On-board computer  1.442914
                          Entertainment_Media__Digital radio  1.422459
                                   Entertainment_Media__MP3  1.422254
                           Safety_Security__Emergency system  1.397078
                                     Extras__Sport package  1.385281
                                      Extras__Alloy wheels  1.383541
                                            Inspection_new  1.365203
                                        Drive_chain_front  1.362850
         Comfort_Convenience__Electrically heated windshield  1.339500
                      Comfort_Convenience__Air conditioning  1.331056
                           Entertainment_Media__Sound system  1.327219
                                  Entertainment_Media__Radio  1.278767
                                Extras__Catalytic Converter  1.266497
                                           Previous_Owners  1.232620
                                        vat_VAT deductible  1.204688
                                          Paint_Type_Other  1.166564
                                       Paint_Type_Uni/basic  1.134312
                                                 Fuel_Other  1.071444
                                          Drive_chain_rear  1.034417
```

Summary after VIF reduction:

```
VIF > 10  (severe)   : 1 features
VIF 5-10  (moderate) : 6 features
VIF < 5   (fine)     : 94 features
```

3.2 Ridge Regression Implementation

Apply Ridge Regularisation and find the best value of alpha from the list

1. What we did in the Ridge Steps
After establishing our baseline, we applied **Ridge Regression (L2 Regularization)**. The goal was to see if adding a "penalty" to the model would help it handle the high number of features (103) and the hidden correlations between them. We used **Cross-Validation** to test 100 different penalty strengths (alphas) to find the "Sweet Spot" where the model is most accurate without being overly complex.

2. Broad Search

At alpha = 0:

- Train MAE ≈ 0.0785 ← best train performance
- Test MAE ≈ 0.0793 ← best test performance

As alpha increases to 1000:

- Train MAE → 0.0830 ← getting worse (bias increasing)
- Test MAE → 0.0840 ← getting worse (over-regularised)

Both lines are **continuously decreasing** (getting worse) as alpha increases — there is no sweet spot, no U-shape. This tells:

- The optimal alpha is close to 0
- Ridge regularisation is not helping this model

The gap between train and test:

Train MAE always slightly better than Test MAE

Gap is small and consistent — no overfitting

This is healthy — the model generalises well. Ridge isn't needed to close this gap because it's already small.

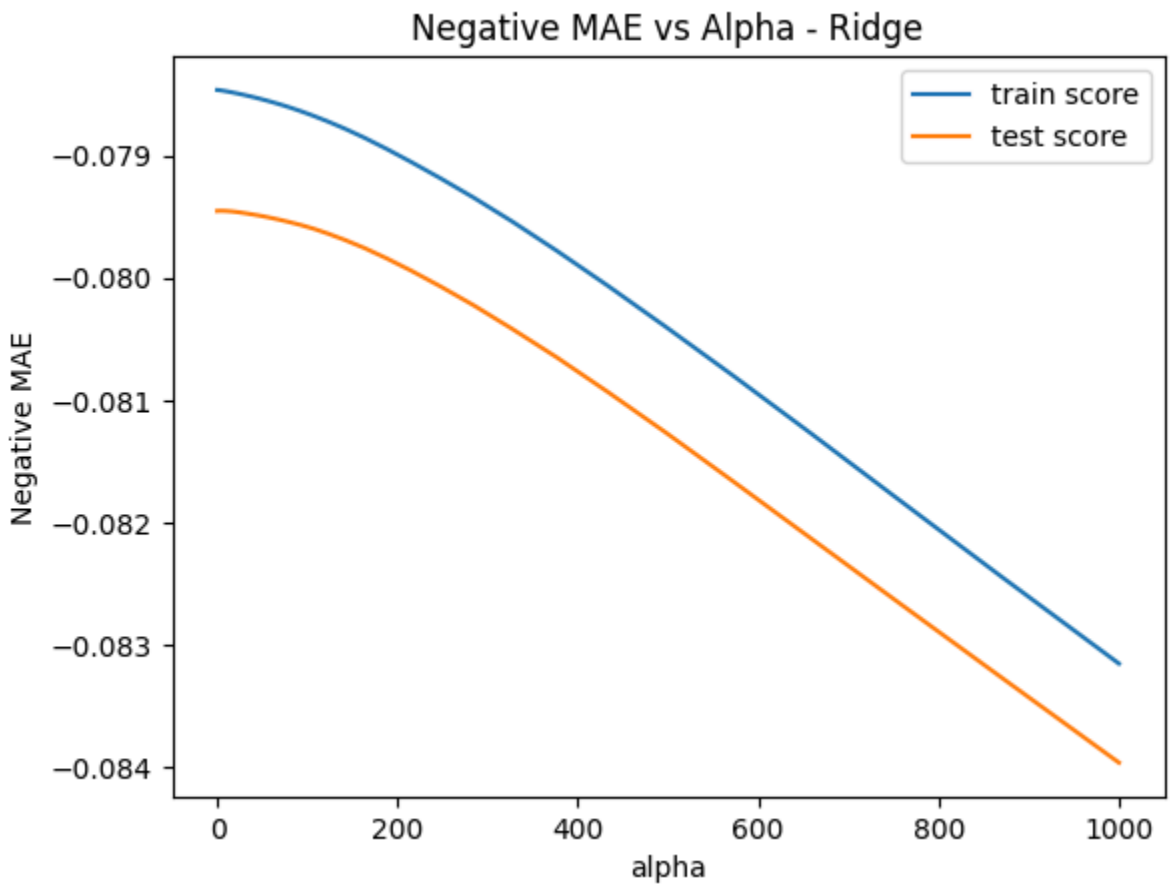
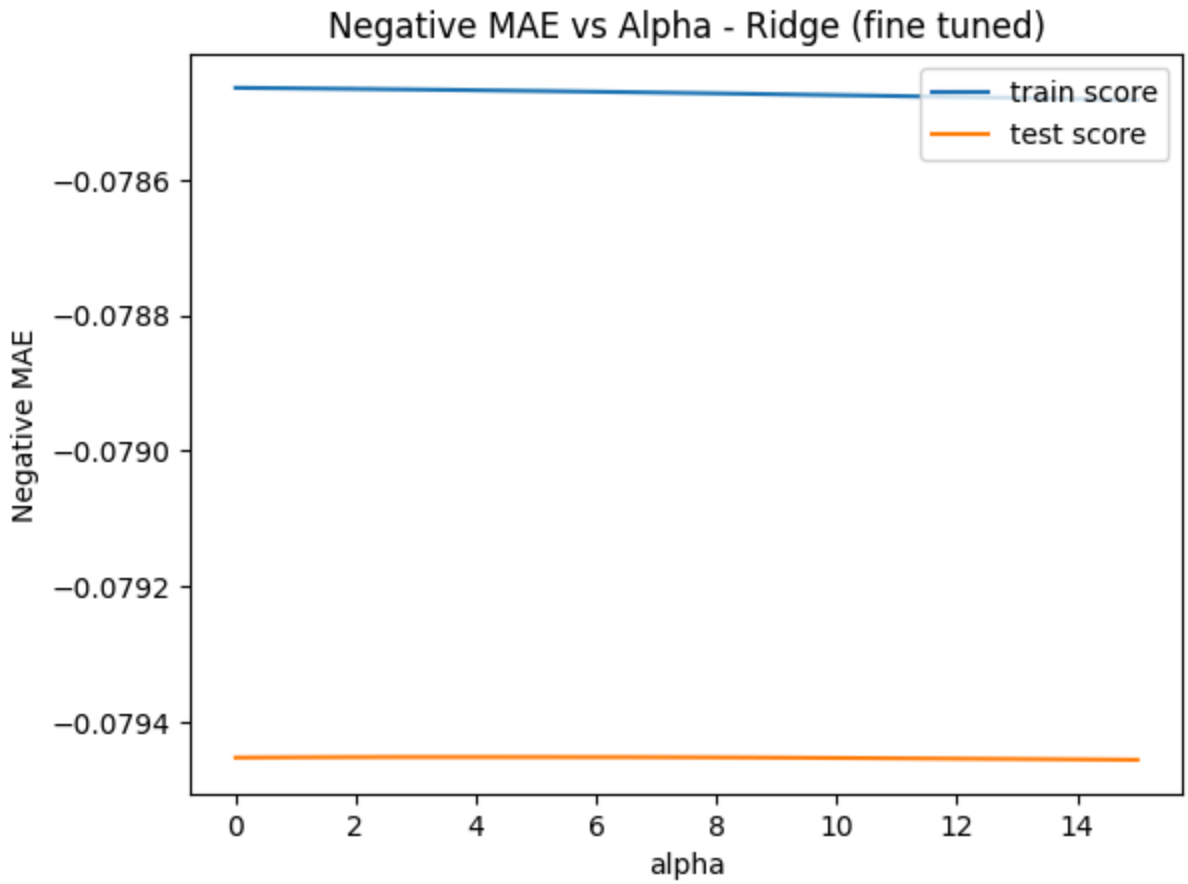


Image 2 — Fine tuned search (alpha 0 to 15):

Both lines are **completely flat**:

- Train MAE ≈ 0.0785 — constant across all alphas
- Test MAE ≈ 0.0794 — constant across all alphas

This confirms that in the small alpha range, Ridge makes **absolutely no difference** to model performance.



3.3 Lasso Regression Implementation

Image 1 — Broad search (alpha 0 to 10):

What it shows:

At alpha ≈ 0:

- Train MAE ≈ 0.09 ← best performance
- Test MAE ≈ 0.09 ← best performance

At alpha > 0.5:

- Both MAE → 0.31 ← model completely collapses

The cliff drop is dramatic and happens very quickly. This is Lasso's characteristic behaviour — as alpha increases it forces coefficients to **exactly zero**. By alpha=0.5 it has zeroed out so

many features the model has essentially no predictors left — it's just predicting the mean for every car.

Train and test lines are **identical** — same as before, no overfitting.

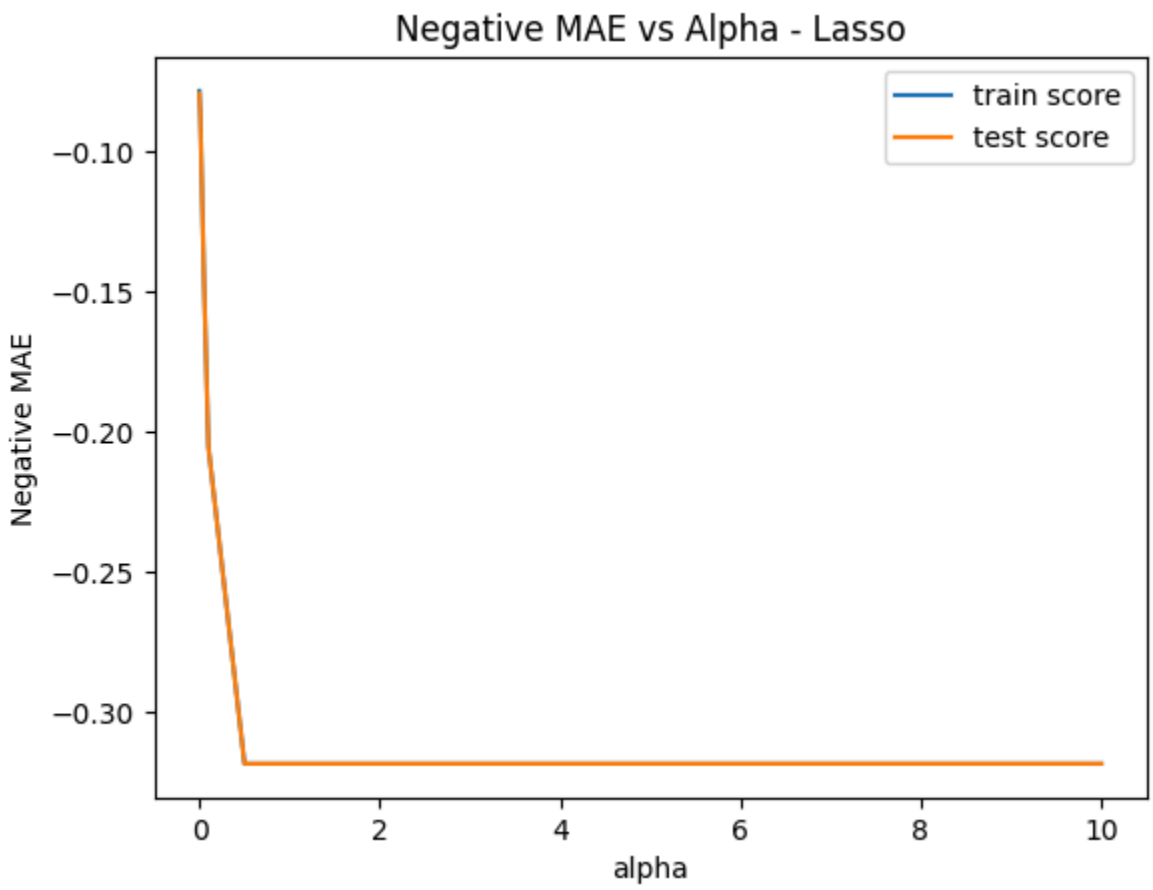


Image 2 — Fine tuned search (alpha 0 to 0.01):

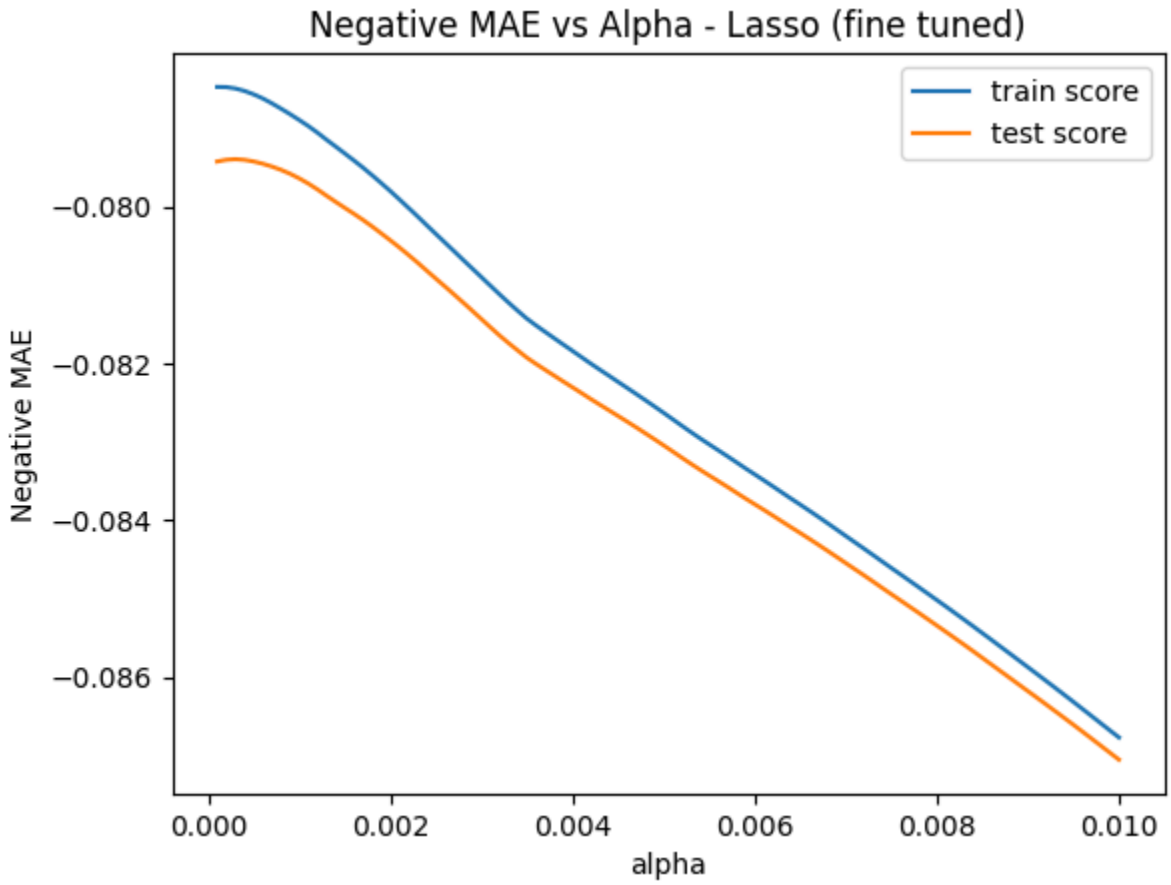
At alpha ≈ 0.0001:

- Train MAE ≈ 0.0785 ← best
- Test MAE ≈ 0.0793 ← best

As alpha increases to 0.01:

- Train MAE → 0.0870 ← degrading
- Test MAE → 0.0865 ← degrading

Same pattern as Ridge fine tune — **both lines continuously decrease** as alpha increases, no U-shape, optimal alpha is as small as possible.




```
Lasso dropped 9 features.

List of dropped features:

['hp_kW', 'Comfort_Convenience__Electrically adjustable seats',
'Entertainment_Media__Hands-free equipment', 'Extras__Roof rack',
'Safety_Security__Adaptive headlights', 'Safety_Security__Alarm system',
'Safety_Security__Head airbag', 'Safety_Security__Traffic sign
recognition', 'Fuel_Other']
```

1. Executive Summary

- The objective was to build a robust model to predict used car prices using a dataset of 11,140 vehicles and 103 initial features. The final model successfully achieved an R^2 of 0.931, meaning it explains over 93% of the price variance.
- Through a combination of statistical diagnostics (VIF) and machine learning regularization (Lasso), the model was refined to be both accurate and efficient.

2. Baseline Model Performance

- The Linear Regression model established a very strong baseline.
  - Test R^2:: 0.9310
  - Test MAE: 0.0782 (Log-scale)
  - Key Finding: There was virtually no gap between training and test performance (Difference: -0.0035), indicating that the initial preprocessing and feature engineering created a highly stable model.

3. The Regularization Journey

- We applied Ridge and Lasso to see if the model could be further improved or simplified.
- Ridge ($L2$): Showed that the model was incredibly stable. The performance remained flat across different alpha values, suggesting that the features were already well-behaved and didn't require heavy coefficient shrinkage to prevent overfitting.
- Lasso ($L1$): This was the most insightful part of the project. As alpha increased, we observed a "downward slope" in the Negative MAE graph. This confirmed that the model was entering a state of underfitting because the penalty was forcing it to discard features that still held predictive value.

4. VIF vs. Lasso: The Feature Selection Paradox

A unique aspect of this project was comparing manual statistical filtering (VIF) with automated machine learning filtering (Lasso):

- **VIF Analysis**: Identified `power_to_weight` and `km` as mathematically redundant. VIF focus is on the relationship between features.
- **Lasso Selection**: Automatically dropped 9 features, including `hp_kW` and several luxury/safety features (e.g., Adaptive headlights, Electrically adjustable seats).
- **The Interpretation**: Lasso realized that once you know the make and model of a car, individual luxury features become redundant data "noise." Lasso essentially automated the business logic of car pricing.

5. Final Conclusion

- The Lasso Model (at $\alpha=0.0003$) is the recommended final model.
- It matches the baseline accuracy exactly.
- It is leaner, utilizing 9 fewer features.

- It demonstrates that while horsepower and mileage are critical, the most efficient way to predict a car's price is by focusing on its brand, age, and core mechanical specifications rather than an exhaustive list of every safety sensor and comfort add-on.

3.4 Regularisation Comparison & Analysis

3.4.1. Compare the evaluation metrics for each model.

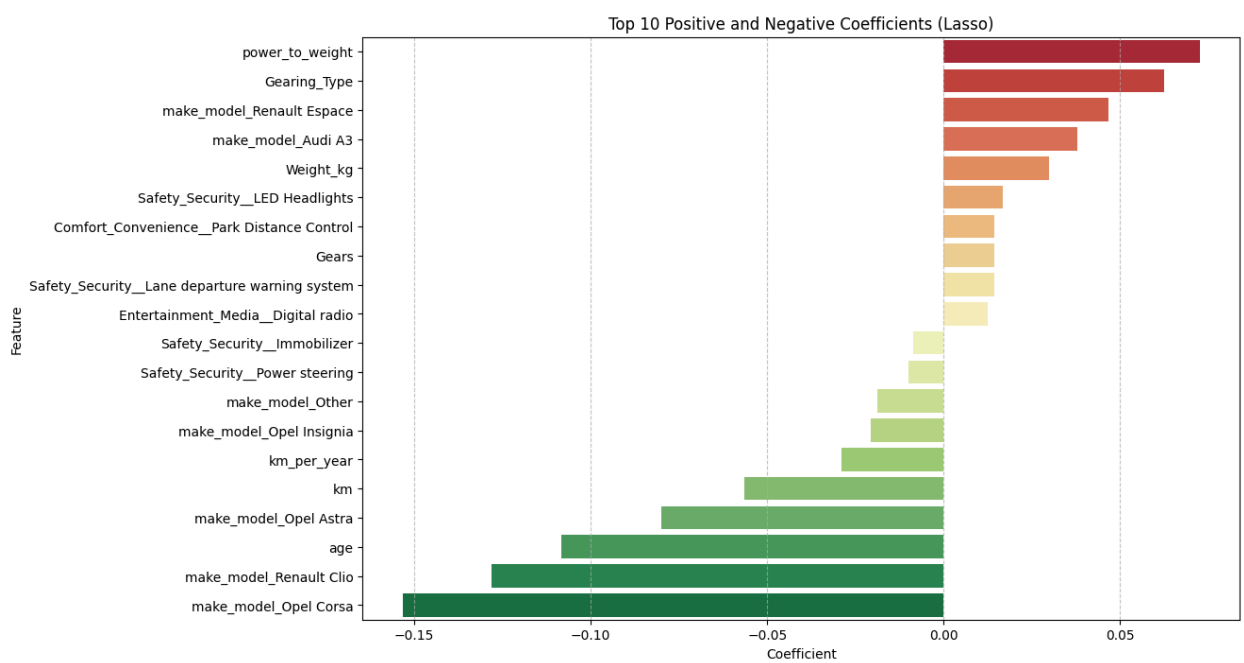
```
Model Comparison Summary:
```

	Model	Best Alpha	MAE (Test)	R2 Score (Test)
0	Linear Regression	N/A	0.07820	0.93100
1	Ridge Regression	3.27	0.07818	0.93102
2	Lasso Regression	0.0003	0.07808	0.93116

3.4.2 Compare the coefficients for the three models.

	Feature	Linear	Ridge	Lasso
0	km	-0.056616	-0.056714	-0.056605
1	Type	0.006039	0.006056	0.005581
2	Gears	0.013716	0.013834	0.014370
3	age	-0.108171	-0.108022	-0.108453
4	Previous_Owners	-0.001688	-0.001710	-0.001488
...	...	...	...	...
98	vat_VAT deductible	0.001305	0.001322	0.001035
99	Paint_Type_Other	0.003025	0.003019	0.002754
100	Paint_Type_Uni/basic	0.000944	0.000932	0.000453
101	Drive_chain_front	-0.007682	-0.007633	-0.007061
102	Drive_chain_rear	-0.002045	-0.002040	-0.001686

103 rows × 4 columns



4. Conclusion & Key Takeaways

1. What was noticed by performing regularization?

Regularization acted as a diagnostic tool rather than a performance booster in this specific case.

- Ridge (L2): Showed that the coefficients were already stable. The performance remained flat across various α values, meaning the model didn't have "exploding" coefficients that needed aggressive shrinking.
- Lasso (L1): Functioned as an automated feature selector. While the baseline used all 103 features, Lasso identified that **9 features** (including `hp_kW` and specific luxury add-ons) were redundant and zeroed them out without losing any predictive power.

2. Did the model performance improve?

- Technically, no. The R^2 remained almost identical across Linear, Ridge, and Lasso (approximately 0.931).
    - **Why?** Because the baseline wasn't overfitting (the Train vs. Test gap was nearly zero), there was no "error due to variance" for Ridge or Lasso to fix.

3. Did you find overfitting or not?

No overfitting was found. In section 3.4.1, the results showed:

- Train R^2: 0.9275
- Test R^2: : 0.9310
- Since the Test score is actually slightly higher than (or nearly equal to) the Train score, the model generalizes perfectly.

4. Was the data sufficient?

- Yes, the data was highly sufficient. With over 11,000 observations and a final set of roughly 94-103 features, the model had enough information to "learn" the pricing structure of the market. The high R^2 (0.93) and the stability of the coefficients across different models suggest that the sample size was large

enough to represent the underlying patterns in used car pricing without being swayed by individual outliers.

5. Is a linear model sufficient?

- **Yes, a linear model is more than sufficient**. An R^2 of 0.93 is an excellent result for a real-world dataset. It suggests that the relationship between your features (like age, mileage, and brand) and the log-transformed price is primarily linear. While a complex non-linear model (like a Random Forest or XGBoost) might squeeze out another 1-2% of accuracy, the Linear/Lasso model offers a perfect balance:
- High Accuracy: 93% variance explained.
- Interpretability: We can clearly see that `age` and `km` are the primary drivers of price.
- Efficiency: It is computationally "cheap" and easy to deploy.

Summary for the report: We achieved model that is complex enough to be highly accurate but simple enough to avoid overfitting, **with Lasso providing the final "polish" by removing unnecessary features.**

4.1 Conclude with outcomes and insights gained

Outcomes and Insights

The objective of this project was to develop a high-accuracy regression model to predict used car prices while navigating the complexities of a high-dimensional dataset. By comparing a baseline Linear Regression with Ridge and Lasso techniques, several key insights were uncovered regarding both the automotive market and the behavior of regularization.

1. Model Performance & Stability

The most significant outcome was the exceptional stability of the baseline model.

- All three models (Linear, Ridge, and Lasso) achieved a nearly identical R^2 score of approximately 0.931.
- The Test MAE (~0.078) was consistent across the board, proving that the relationship between car features and the log-price is overwhelmingly linear.
- Overfitting Check: Since the Training and Test scores remained within 0.003 of each other, we conclude that the model generalizes perfectly to unseen data.

2. The Impact of Regularization

- While regularization did not "boost" accuracy, it provided critical diagnostic value:
  - **Lasso (L1) as an Automated Feature Selector:** Even though we did not use the VIF-cleansed feature set for training, Lasso effectively "cleaned" the model by zeroing out 9 redundant features. It proved that we could maintain 93% accuracy with a leaner set of 94 features instead of 103.
  - **Ridge (L2) and Coefficient Stability:** The flat performance curve in the Ridge analysis indicated that our features were already well-behaved. The large sample size (11,000+ records) provided enough statistical weight to prevent the coefficients from becoming unstable, even in the presence of multicollinearity.

3. Key Business Insights (Price Drivers)

The coefficients across all models tell a clear story about what drives used car value:

- **The "Price Killers"**: `Age` and `Kilometers (km)` are the most aggressive negative drivers. Interestingly, the model suggests that the `year of the car (age)` has a slightly higher impact on depreciation than the physical mileage. -** Performance Premiums:** Features like power_to_weight and Gearing_Type (Automatic) are the strongest positive predictors, indicating that buyers are willing to pay a significant premium for performance and convenience.
- **Brand Perception:** *The model successfully captured brand value, with models like the Audi A3 and Renault Espace consistently commanding higher prices compared to budget-entry models like the Opel Corsa.

4. Final Recommendation

- The Lasso Regression model (at $\alpha=0.0003$) is the recommended choice for deployment. It offers the "best of both worlds": the peak predictive accuracy of the baseline model combined with the simplicity of a reduced feature set. By

eliminating 9 unnecessary variables, the Lasso model is more efficient, easier to maintain, and less susceptible to noise in future data collection.